CS323 Documentation

About 2 pages

1. Problem Statement

Assignment 2: We need to write a syntax analyzer for Rat26F so the tokens can be converted into parse trees and syntax errors can be caught.

2. How to use your program

For a file containing Rat26F source code named 'input.txt', the following command will generate a file named 'output.txt' with the results of the syntax analyzer:

```
python assignment2.py input.txt output.txt
```

If a syntax error is found, the error message will automatically be added to the end of the output file.

3. Design of your program

The syntax analyzer works by using a Recursive Descent Parser (RDP) defined as a Python class. An RDP implements the programming language's grammar rules as individual functions. Our design implements functions such as **`_lexer()`** *to move the analyzer to the next token and* **`_token()`** *and* **`_lexeme()`** *to get the current token and lexeme. We also have* **`_syntax_error()`** *functions defined to report information about a detected syntax error such as the expected/required token and the token that was received instead, as well as the position in the source code where the syntax error is.*

When a single RDP function calls **`_lexer()`***, the syntax analyzer consumes an input and moves onto the next token in the source code.*

For example, the grammar rule for a Rat26F Qualifier (R8) is implemented as a member function of the RDP class. At the beginning of the function, the current token is expected to be (and should be) a Qualifier token. The Qualifier token is checked to see if it is a valid Qualifier token (which means it has the token type 'keyword' and its lexeme is either 'integer', 'boolean', or 'real'). If this check passes,

`_lexer()` *is called, consuming an input. If the check fails, the token is invalid and a syntax error is thrown.*

The beginning of each RDP function (e.g. a single grammar rule) expects the correct state, and therefore the current token must be set correctly before calling an RDP function. Assuming this is the case (which it should) the RDP functions will check that the current token is appropriate for the current context and position. For example, a declaration works by checking if the current token is a valid qualifier, then consuming an input to move to the next token (using **`_lexer()`***), then calls the Identifier's RDP function which will check if the current token that we just moved to is a valid Identifier.*

Recursion is used to collectively advance the syntax analyzer to the next tokens, which is achieved by having the individual RDP functions call other RDP functions and having these individual RDP functions call **`_lexer()`**.

In some cases, we need to move to another parsing state without consuming any input. This occurs when using the rule for the 'Empty' token type, which acts as ε (epsilon), the symbol used to indicate state transition without input consumption. For example, the rule for OptDeclarationList is

<Opt Declaration List> ::= <Declaration List> | <Empty>

which means consumption of tokens for DeclarationList is optional (as we have the option to use <Empty> a.k.a ε *). This allows syntax analysis to work for Rat26F source code with no declarations. In effect, this means moving to other grammar rules without calling* **`_lexer()`**. *The main benefit of this is that the state of the syntax analyzer is not messed up by moving one token forward more than it should. Instead, the syntax analyzer would correctly use another rule to parse the current token.*

A user of our RDP class simply initializes an instance of the class with the input source code, and calls the **`parse()`** *method on the instance, and the Recursive Descent Parser class takes care of the rest.*

4. **Any Limitation**
   <All features are running according to the assignment but you limit your program due to resource limitations, such as Maximum number of lines in the source code, size of the identifier, integer etc. ***Say 'None' if there is no limitation****>*

   None

5. **Any shortcomings**
   <Anything you could NOT implement although that is required by the Assignment. ***Say 'None' if there is no shortcoming****>*

   None